

Supplementary Material:
PolyEdge: Autonomous Prediction Market Trading
with Bounded AGI Safety, Evolutionary Strategy Composition,
and Adversarial Decision Validation

Muchammad Fikri Izzuddin
Independent Researcher
fikri@berkahkarya.com

May 5, 2026

Contents

1	Extended Proofs and Bounds	S2
1.1	Promotion Pipeline Prevents Catastrophic Deployment	S2
1.2	Kelly Criterion for Binary Prediction Markets	S2
1.3	Thompson Sampling Allocation	S2
2	StrategyGenome Grammar	S3
3	Extended Experiment Data	S3
3.1	Per-Strategy Aggregated Breakdown	S4
4	Code Listings	S5
4.1	Autonomous Promotion Gate	S5
4.2	Local Debate Fallback Engine	S5
4.3	StrategyGenome Chromosome Structure	S6
4.4	Fitness Score Computation	S6
5	System Logs	S7
5.1	Experiment Kill	S7
5.2	Trade Attempt Rejection	S7
5.3	Debate Verdict	S7
6	Additional References	S7

1 Extended Proofs and Bounds

1.1 Promotion Pipeline Prevents Catastrophic Deployment

Proposition 1. *If the promotion pipeline requires minimum Sharpe ratio $S_{\min} > 0$ for PAPER→LIVE transition, then no strategy with negative drift can be promoted to live trading.*

Proof. By definition, the Sharpe ratio is:

$$S = \frac{\mathbb{E}[R] - R_f}{\sigma_R} \quad (1)$$

where R is the strategy return, R_f is the risk-free rate, and σ_R is the standard deviation of returns.

If $S > 0$, then $\mathbb{E}[R] > R_f$, which means the strategy has positive expected excess return. A strategy with negative drift has $\mathbb{E}[R] < 0 < R_f$, so $S < 0$. The gate $S \geq S_{\min} > 0$ therefore excludes all negative-drift strategies from LIVE promotion. $\square$

Corollary 1. *The PolyEdge promotion pipeline with thresholds $MIN_SHARPE_PAPER = 0.5$, $MIN_WIN_RATE = 0.45$, and $MAX_DRAWDOWN = 0.20$ enforces a minimum risk-adjusted return at least $0.5\sigma_R$ above the risk-free rate, while capping maximum drawdown at 20%.*

1.2 Kelly Criterion for Binary Prediction Markets

Proposition 2. *For a binary prediction market with contract price p (reflecting implied probability) and true probability π , the Kelly-optimal bet size is:*

$$f^* = \frac{\pi(1-p) - (1-\pi)p}{p(1-p)} \quad (2)$$

Proof. In a binary market, buying YES at price p yields payoff \$1 with probability π and \$0 with probability $1 - \pi$. The expected log-growth is:

$$G(f) = \pi \ln(1 + f/p - f) + (1 - \pi) \ln(1 - f) \quad (3)$$

Maximizing with respect to f :

$$\frac{dG}{df} = \frac{\pi(1-p)}{p + f(1-p)} - \frac{1-\pi}{1-f} = 0 \quad (4)$$

Solving for f yields $f^* = \frac{\pi-p}{p(1-p)} \cdot p = \frac{\pi(1-p)-(1-\pi)p}{p(1-p)}$. $\square$

Practical implementation: PolyEdge uses fractional Kelly ($f = 0.25 \cdot f^*$ for the “safe” risk profile and $f = 0.50 \cdot f^*$ for “normal”) to reduce variance at the cost of expected growth rate.

1.3 Thompson Sampling Allocation

The bankroll allocator rank-strategies using Thompson sampling from a Beta distribution:

$$S_k \sim \text{Beta}(\alpha_k + 1, \beta_k + 1) \quad (5)$$

where α_k is the cumulative win count and β_k is the cumulative loss count for strategy k . This naturally balances exploration (trying under-sampled strategies) with exploitation (allocating to high-performing strategies).

2 StrategyGenome Grammar

The complete BNF grammar for StrategyGenome chromosome specification:

Listing 1: StrategyGenome BNF Grammar

```

1 <genome> ::= <perception> <cognition> <execution> <risk> <meta>
2
3 <perception> ::= <data-sources> <extractors> <timeframes> <aggregation>
4 <data-sources> ::= "polymarket_clob" | "binance_ws" | "open-meteo" | ...
5 <extractors> ::= "price_velocity" | "orderbook_imbalance" | "volume_weighted" | ...
6 <timeframes> ::= "1m" | "5m" | "15m" | "1h" | "4h" | "1d"
7 <aggregation> ::= "weighted_average" | "majority_vote" | "bayesian_fusion"
8
9 <cognition> ::= <entry-logic> <exit-logic> <market-selector>
10 <entry-logic> ::= <trigger-type> <conditions> <conjunction> <min-confidence>
11 <trigger-type> ::= "threshold_cross" | "pattern_match" | "statistical_arbitrage"
12 | "event_driven" | "momentum_breakout"
13 <conditions> ::= {<indicator> <operator> <value> <weight>}+
14 <conjunction> ::= "AND" | "OR" | "weighted_score"
15 <min-confidence> ::= FLOAT in [0.0, 1.0]
16
17 <exit-logic> ::= <trigger-type> <profit-target> <stop-loss> <max-hold>
18 <profit-target> ::= FLOAT | None
19 <stop-loss> ::= FLOAT | None
20 <max-hold> ::= FLOAT (hours)
21
22 <market-selector> ::= <criteria> <scoring-function> <max-concurrent>
23 <criteria> ::= "high_volume" | "short_settlement" | "low_spread" | ...
24 <scoring-function> ::= "weighted_composite_score"
25
26 <execution> ::= <order-type> <slippage-tol> <speed-target> <retry> <atomic>
27 <order-type> ::= "limit" | "market" | "fok" | "ioc" | "post_only"
28 <slippage-tol> ::= FLOAT in [0.0, 0.05]
29 <speed-target> ::= INT (milliseconds)
30 <retry> ::= "exponential_backoff" | "immediate" | "abandon"
31 <atomic> ::= BOOL
32
33 <risk> ::= <sizing-model> <kelly-frac> <max-position> <max-exposure>
34 <daily-dd> <weekly-dd> <correlation-aware>
35 <sizing-model> ::= "kelly_fraction" | "fixed_fraction" | "volatility_targeted"
36 <correlation-aware> ::= BOOL
37
38 <meta> ::= <self-opt> <tune-freq> <adapt-speed> <crossover-elig>
39 <mutation-rate> <next-mutation-target>
40 <self-opt> ::= BOOL
41 <tune-freq> ::= "every_trade" | "hourly" | "daily" | "weekly"
42 <adapt-speed> ::= "conservative" | "moderate" | "aggressive"
43 <crossover-elig> ::= BOOL
44 <mutation-rate> ::= FLOAT in [0.01, 0.50]

```

3 Extended Experiment Data

Table 1: All 25 experiments with full metrics

Strategy	Status	Trades	Win Rate	Sharpe	Max DD	PnL
general_scanner backtest	—	—	—	—	—	—
general_scanner backtest	—	—	—	—	—	—
general_scanner backtest	—	—	—	—	—	—

Continued on next page

Strategy	Status	Trades	Win Rate	Sharpe	Max DD	PnL
general_scanner	backtest	–	–	–	–	–
general_scanner	backtest	–	–	–	–	–
general_scanner	backtest	–	–	–	–	–
general_scanner	backtest	–	–	–	–	–
general_scanner	backtest	–	–	–	–	–
general_scanner	backtest	–	–	–	–	–
btc_oracle	backtest	–	–	–	–	–
btc_oracle	backtest	–	–	–	–	–
btc_oracle	backtest	–	–	–	–	–
unknown	draft	–	–	–	–	–
unknown	draft	–	–	–	–	–
unknown	draft	–	–	–	–	–
null	draft	–	–	–	–	–
null	draft	–	–	–	–	–
null	draft	–	–	–	–	–
general_scanner	shadow	–	–	–	–	–
general_scanner	shadow	–	–	–	–	–
general_scanner	shadow	–	–	–	–	–
btc_oracle	shadow	–	–	–	–	–
null	shadow	–	–	–	–	–
unknown	shadow	–	–	–	–	–
null	shadow	–	–	–	–	–
unknown	shadow	–	–	–	–	–

Note: Individual experiment metrics are available in the supplementary data file `data/experiments.json`. Backtest experiments have not yet produced live trading metrics. Shadow experiments show aggregate data but individual metrics are not yet statistically significant ($n < 20$ trades per experiment, below the minimum threshold for fitness evaluation).

3.1 Per-Strategy Aggregated Breakdown

Table 2: Aggregate metrics by strategy type across all experiments

Strategy	Trades	Win Rate	Avg PnL/Trade	Total PnL	Markets
general_scanner	105	34.3%	–\$4.12	–\$432.60	87
btc_oracle	35	40.0%	–\$3.26	–\$114.10	28
unknown	15	26.7%	–\$5.02	–\$75.34	17
Total	162	35.7%	–\$3.84	–\$622.04	132

4 Code Listings

4.1 Autonomous Promotion Gate

The core promotion decision logic from `autonomous_promoter.py`, showing the health-based evaluation criteria for PAPER→LIVE transition:

Listing 2: Paper-to-Live promotion criteria (`autonomous_promoter.py`, lines 80–110)

```
1 def _check_paper_criteria_from_health(  
2     self, exp: ExperimentRecord, health: dict  
3 ) -> tuple[bool, list[str]]:  
4     """Evaluate paper->live promotion using current health metrics."""  
5     reasons = []  
6     trades = health.get("total_trades", 0)  
7     win_rate = health.get("win_rate", 0.0)  
8     sharpe = health.get("sharpe", 0.0)  
9     max_dd = health.get("max_drawdown", 0.0)  
10  
11     if trades < self.MIN_TRADES_PAPER:  
12         reasons.append(f"trades {trades} < {self.MIN_TRADES_PAPER}")  
13     if win_rate < self.MIN_WIN_RATE_PAPER:  
14         reasons.append(f"win_rate {win_rate:.1%} < "  
15             f"{self.MIN_WIN_RATE_PAPER:.1%}")  
16     if sharpe < self.MIN_SHARPE_PAPER:  
17         reasons.append(f"sharpe {sharpe:.2f} < "  
18             f"{self.MIN_SHARPE_PAPER:.2f}")  
19     if max_dd > self.MAX_DRAWDOWN_PAPER:  
20         reasons.append(f"dd {max_dd:.1%} > "  
21             f"{self.MAX_DRAWDOWN_PAPER:.1%}")  
22  
23     passed = len(reasons) == 0  
24     return passed, reasons
```

The `ExperimentRecord` transitions through stages DRAFT → SHADOW → PAPER → LIVE_PROMOTED → RETIRED. Each transition requires satisfying all criteria; any single failure blocks promotion.

4.2 Local Debate Fallback Engine

When the external MiroFish debate service is unavailable, PolyEdge falls back to a local debate engine that evaluates signals using heuristic analysis:

Listing 3: Local debate fallback (`services/mirofish_service.py`, adapted)

```
1 class LocalDebateEngine:  
2     """Fallback debate engine when MiroFish is unavailable.  
3     Uses rule-based bull/bear synthesis for basic signal validation.  
4     """  
5     def evaluate_signal(self, signal: TradingSignal) -> DebateVerdict:  
6         bull_args = self._bull_case(signal)  
7         bear_args = self._bear_case(signal)  
8         verdict = self._adjudicate(bull_args, bear_args, signal)  
9         return verdict  
10  
11     def _bull_case(self, signal: TradingSignal) -> list[str]:  
12         args = []  
13         if signal.confidence and signal.confidence >= 0.7:  
14             args.append(f"High confidence ({signal.confidence:.0%})")  
15         if signal.edge_pct and signal.edge_pct >= 0.05:  
16             args.append(f"Edge detected ({signal.edge_pct:.1%})")  
17         if self._trending_favorable(signal):  
18             args.append("Market trend favorable")  
19         return args  
20  
21     def _bear_case(self, signal: TradingSignal) -> list[str]:
```

```

22     args = []
23     if signal.confidence and signal.confidence < 0.5:
24         args.append(f"Low confidence ({signal.confidence:.0%})")
25     if signal.edge_pct and signal.edge_pct < 0.03:
26         args.append(f"Insufficient edge ({signal.edge_pct:.1%})")
27     if self._counter_indicator(signal):
28         args.append("Counter-indicator present")
29     return args
30
31     def _adjudicate(self, bull, bear, signal) -> DebateVerdict:
32         if len(bull) > len(bear) and signal.confidence and signal.confidence >= 0.6:
33             return DebateVerdict(should_trade=True, confidence=0.65)
34         return DebateVerdict(should_trade=False, confidence=0.35)

```

4.3 StrategyGenome Chromosome Structure

The Pydantic model for StrategyGenome, showing the five-chromosome structure:

Listing 4: StrategyGenome Pydantic model (domain/genome/models.py)

```

1 class StrategyGenome(BaseModel):
2     genome_id: str = Field(default_factory=lambda: str(uuid4()))
3     strategy_name: str
4     archetype: str
5     version: str = "1.0.0"
6     stage: Literal["DRAFT", "SHADOW", "PAPER",
7                   "LIVE", "BREEDING", "LEGEND", "GRAVEYARD"] = "DRAFT"
8     lineage: LineageData = Field(default_factory=LineageData)
9     chromosomes: dict = Field(...)
10    fitness_metrics: FitnessMetrics = Field(
11        default_factory=FitnessMetrics)
12    chromosome_performance_history: dict[str, list[float]] = Field(
13        default_factory=dict)
14    death_certificate: Optional[dict] = None
15    created_at: datetime = Field(default_factory=datetime.utcnow)
16    updated_at: datetime = Field(default_factory=datetime.utcnow)

```

4.4 Fitness Score Computation

The composite fitness function that drives evolutionary selection:

Listing 5: Fitness calculation (domain/evolution/fitness.py)

```

1 def calculate_fitness(metrics: FitnessMetrics) -> float:
2     """Calculate fitness score from 0.0 (worst) to 1.0 (best).
3     Requires at least 20 trades for meaningful evaluation."""
4     if metrics.total_trades < 20:
5         return 0.0
6
7     score = (
8         (normalize(metrics.sharpe_ratio, -3, 3) * 0.30) +
9         (metrics.win_rate * 0.20) +
10        (normalize(metrics.profit_factor, 0, 5) * 0.15) +
11        ((1.0 - metrics.max_drawdown_pct) * 0.15) +
12        (normalize(metrics.alpha_per_trade, -1, 1) * 0.10) +
13        (metrics.capital_rotation_efficiency * 0.10)
14    )
15    return max(0.0, min(1.0, score))

```

The weight allocation (30% Sharpe, 20% win rate, 15% profit factor, 15% drawdown penalty, 10% alpha, 10% capital rotation) was chosen to prioritize risk-adjusted returns while maintaining diversity pressure through the capital rotation efficiency term.

5 System Logs

Anonymized log excerpts showing key system behaviors:

5.1 Experiment Kill

Listing 6: Strategy health kill event (AGI log, 2026-05-04)

```
1 2026-05-04 02:15:33 [HEALTH] Strategy general_scanner_v3 health assessment:
2   status=KILLED, reason="7-day win_rate 0.28 < threshold 0.40"
3   total_trades=23, pnl=-45.67, max_drawdown=0.34
4   ACTION: Disabling strategy config and marking experiment RETIRED
```

5.2 Trade Attempt Rejection

Listing 7: Risk manager rejection (RISK log, 2026-05-04)

```
1 2026-05-04 14:22:11 [RISK] Trade attempt REJECTED for btc_oracle
2   market="Will BTC close above $95k on May 5?"
3   proposed_size=25.00 USDC, position_fraction=0.125
4   REASON: "Position fraction 0.125 > max_position_fraction 0.08"
5   current_exposure=0.72, max_exposure=0.70
6   ACTION: Trade not submitted to CLOB
```

5.3 Debate Verdict

Listing 8: MiroFish debate verdict (DEBATE log, 2026-05-04)

```
1 2026-05-04 16:45:02 [DEBATE] Signal evaluation completed for btc_oracle
2   market="Will the Fed raise rates in June?"
3   BULL: High confidence (0.78), Edge detected (6.2%), Market trend favorable
4   BEAR: Counter-indicator present, Implied probability suggests overpriced
5   JUDGE VERDICT: should_trade=True, confidence=0.71
6   BULL_ARGS_COUNT=3, BEAR_ARGS_COUNT=1
7   EXECUTION: Signal approved for capital deployment
```

Note: These logs are illustrative reconstructions based on the system’s logging format and observed behavior patterns. Actual logs contain market-specific identifiers and wallet addresses that have been anonymized.

6 Additional References

- **OpenAI Gym** [Brockman et al., 2016]: Standardized toolkit for developing and comparing reinforcement learning algorithms. PolyEdge’s promotion pipeline evaluation is inspired by Gym’s environment API for episodic evaluation.
- **Papers With Code** (paperswithcode.com): Used for benchmarking strategy fitness metrics against published baselines in algorithmic trading and time-series forecasting.
- **Pydantic** (pydantic-docs.helpmanual.io): Data validation framework used for all StrategyGenome models, providing type-safe mutation and crossover operations.
- **SQLAlchemy 2.0** (sqlalchemy.org): Object-relational mapping for experiment persistence, supporting concurrent access from the promotion daemon and strategy health checker.

- **Polymarket CLOB SDK** (docs.polymarket.com): Python client library for interacting with the Polymarket Central Limit Order Book, used for order placement and market data retrieval.

References

Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym, 2016.